JUnit 5 • Mockito • Spring Testing

Complete In-Depth Guide – From Scratch

JUnit 5 Fundamentals • All Assertions • Parameterised Tests

Mockito Mocks & Stubs • Argument Matchers • Spy & Capture

@SpringBootTest • @WebMvcTest • @DataJpaTest • MockMvc

Testcontainers • WireMock • Test Slices • BDD Style

Clear Definitions • Usage • Full Code Examples

Table of Contents

CHAPTER 1

JUnit 5 Fundamentals

Architecture, Lifecycle, Annotations, Conditional Tests & Nested Tests

1.1 What is JUnit 5?

JUnit 5

JUnit 5 is the fifth major version of the JUnit testing framework for Java. Unlike JUnit 4 (a single monolithic JAR), JUnit 5 is composed of three sub-projects: JUnit Platform (foundation for launching tests), JUnit Jupiter (new programming and extension model), and JUnit Vintage (backward compatibility for JUnit 3/4 tests). 'JUnit 5' typically refers to Jupiter, which introduces lambda support, parameterised tests, nested tests, and a new extension model.

JUnit 4	JUnit 5 (Jupiter)	Purpose
@Before	@BeforeEach	Runs before each test method
@After	@AfterEach	Runs after each test method
@BeforeClass	@BeforeAll	Runs once before all tests in class (static)
@AfterClass	@AfterAll	Runs once after all tests in class (static)
@Ignore	@Disabled	Disables a test
@RunWith	@ExtendWith	Registers test extensions
@Rule	@ExtendWith	Extension mechanism
@Test(expected=)	assertThrows()	Exception assertions
@Test(timeout=)	assertTimeout()	Timeout assertions
No equivalent	@Nested	Inner class test groups
No equivalent	@ParameterizedTest	Data-driven tests
No equivalent	@DisplayName	Human-readable test name

1.2 Maven Setup

Maven dependencies

```
<!-- pom.xml – JUnit 5 + Mockito + AssertJ -->
<dependencies>
<!-- Spring Boot Test – includes JUnit 5, Mockito, AssertJ, MockMvc -->
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>

<!-- Testcontainers for integration tests -->
<dependency>
<groupId>org.testcontainers</groupId>
<artifactId>junit-jupiter</artifactId>
<version>1.19.3</version>
<scope>test</scope>
</dependency>
<dependency>
<groupId>org.testcontainers</groupId>
<artifactId>postgresql</artifactId>
<version>1.19.3</version>
<scope>test</scope>
</dependency>

<!-- WireMock -->
<dependency>
<groupId>com.github.tomakehurst</groupId>
<artifactId>wiremock-jre8</artifactId>
<version>3.0.1</version>
<scope>test</scope>
</dependency>
</dependencies>

<!-- Surefire plugin – runs JUnit 5 tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>3.2.2</version>
</plugin>
```

1.3 Writing Your First Test + Full Lifecycle

Complete JUnit 5 test class with lifecycle

```
// Domain classes under test
public class Calculator {
public int add(int a, int b) { return a + b; }
public int subtract(int a, int b) { return a - b; }
public int multiply(int a, int b) { return a * b; }
public double divide(int a, int b) {
if (b == 0) throw new ArithmeticException("Cannot divide by zero");
return (double) a / b;
}
}
```

```
// JUnit 5 Test Class
class CalculatorTest {

private Calculator calculator;

// ■■ Lifecycle Methods ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@BeforeAll
static void initAll() {
// Runs ONCE before all tests; method must be static
System.out.println("Starting Calculator tests");
}

@BeforeEach
void init() {
// Runs before EACH test method
calculator = new Calculator();
}

@AfterEach
void tearDown() {
// Runs after EACH test method
System.out.println("Test completed");
}

@AfterAll
static void tearDownAll() {
// Runs ONCE after all tests; must be static
System.out.println("All Calculator tests done");
}

// ■■ Test Methods ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Test
@DisplayName("Adding two positive numbers should return their sum")
void addPositiveNumbers() {
int result = calculator.add(3, 7);
assertEquals(10, result);
}

@Test
@DisplayName("Division by zero should throw ArithmeticException")
void divisionByZeroThrowsException() {
ArithmeticException ex = assertThrows(ArithmeticException.class,
() -> calculator.divide(10, 0));
assertEquals("Cannot divide by zero", ex.getMessage());
}

@Test
@Disabled("Temporarily disabled – pending fix for issue #42")
void skippedTest() {
fail("This should not run");
}
```

```
}
```

1.4 Conditional Tests

Conditional test annotations

```
class ConditionalTests {

@Test
@EnabledOnOs(OS.LINUX)
void onlyOnLinux() { /* runs on Linux only */ }

@Test
@DisabledOnOs({OS.WINDOWS, OS.MAC})
void notOnWindowsOrMac() { /* skipped on Windows/Mac */ }

@Test
@EnabledOnJre(JRE.JAVA_17)
void onlyOnJava17() { /* runs only on JDK 17 */ }

@Test
@EnabledIfSystemProperty(named = "env", matches = "ci")
void onlyInCIPipeline() { /* runs only when -Denv=ci */ }

@Test
@EnabledIfEnvironmentVariable(named = "CI", matches = "true")
void onlyWhenCIEnvSet() { /* runs only when CI=true env var */ }

@Test
@EnabledIf("customCondition") // SpEL expression
void conditionalOnSpEL() { }
}
```

1.5 @Nested Tests – Grouping Related Tests

@Nested

@Nested allows you to create inner test classes that represent a sub-group of tests. This is useful for organising tests by method, behaviour, or state – making the test class readable as living documentation. Inner classes share the outer class's @BeforeEach.

@Nested tests – organised by scenario

```
@DisplayName("OrderService tests")
class OrderServiceTest {

private OrderService orderService;

@BeforeEach
void setUp() {
orderService = new OrderService();
}

@Nested
@DisplayName("When creating an order")
class WhenCreatingOrder {
```

```
@Test
@DisplayName("should return PENDING status")
void shouldReturnPendingStatus() {
Order order = orderService.create("cust-1", List.of("item-A"));
assertEquals(OrderStatus.PENDING, order.getStatus());
}

@Test
@DisplayName("should throw exception when items list is empty")
void shouldThrowWhenNoItems() {
assertThrows(IllegalArgumentException.class,
() -> orderService.create("cust-1", Collections.emptyList()));
}
}

@Nested
@DisplayName("When confirming an order")
class WhenConfirmingOrder {

private Order pendingOrder;

@BeforeEach
void createPendingOrder() {
pendingOrder = orderService.create("cust-1", List.of("item-A"));
}

@Test
@DisplayName("should change status to CONFIRMED")
void shouldConfirm() {
orderService.confirm(pendingOrder.getId());
assertEquals(OrderStatus.CONFIRMED, pendingOrder.getStatus());
}

@Test
@DisplayName("should throw when order already cancelled")
void shouldThrowOnCancelledOrder() {
orderService.cancel(pendingOrder.getId());
assertThrows(IllegalStateException.class,
() -> orderService.confirm(pendingOrder.getId()));
}
}
}
```

CHAPTER 2

JUnit 5 Assertions

assertEquals, assertAll, assertThrows, assertTimeout, AssertJ & Hamcrest

2.1 Core JUnit 5 Assertions

Assertion Method	Purpose
assertEquals(expected, actual)	Checks two values are equal
assertNotEquals(unexpected, actual)	Checks two values are NOT equal
assertTrue(condition)	Checks condition is true
assertFalse(condition)	Checks condition is false
assertNull(object)	Checks object is null
assertNotNull(object)	Checks object is not null
assertSame(expected, actual)	Checks both point to same object (==)
assertNotSame(unexpected, actual)	Checks different object references
assertArrayEquals(expected[], actual[])	Checks all array elements are equal
assertIterableEquals(exp, act)	Checks two iterables contain same elements
assertLinesMatch(List, List)	Checks lists of strings match (regex support)
assertThrows(ExType.class, executable)	Checks executable throws specified exception
assertDoesNotThrow(executable)	Checks executable throws no exception
assertTimeout(duration, executable)	Checks executable finishes within duration
assertTimeoutPreemptively(dur, exec)	Aborts and fails if duration exceeded
assertAll(executables...)	Groups assertions – reports all failures
fail(message)	Unconditionally fails a test

Core assertions with examples

```
class AssertionsDemo {

@Test
void coreAssertions() {
// Equality
assertEquals(42, calculator.add(40, 2));
assertEquals(42, calculator.add(40, 2), "Sum should be 42"); // custom msg
assertEquals(3.14, Math.PI, 0.01); // delta for floating point
```

```
// Null checks
assertNull(repository.findById(-1L).orElse(null));
assertNotNull(repository.findById(1L).orElse(null));

// Boolean
assertTrue(list.contains("apple"));
assertFalse(list.isEmpty());

// Collections
assertArrayEquals(new int[]{1,2,3}, new int[]{1,2,3});
assertIterableEquals(List.of("a","b"), List.of("a","b"));
}

@Test
void assertAllGroups() {
User user = userService.findById(1L);
// assertAll runs ALL assertions even if some fail
assertAll("User validation",
() -> assertNotNull(user),
() -> assertEquals("Alice", user.getName()),
() -> assertEquals("alice@example.com", user.getEmail()),
() -> assertTrue(user.isActive())
);
}

@Test
void exceptionTesting() {
// assertThrows returns the exception for further assertions
IllegalArgumentException ex = assertThrows(
IllegalArgumentException.class,
() -> userService.createUser(null));
assertNotNull(ex.getMessage());
assertTrue(ex.getMessage().contains("name"));

// assertDoesNotThrow
assertDoesNotThrow(() -> userService.createUser("Alice"));
}

@Test
void timeoutTesting() {
// Fails if execution takes more than 2 seconds
assertTimeout(Duration.ofSeconds(2), () -> {
reportService.generateReport();
});
// assertTimeoutPreemptively aborts execution if too slow
assertTimeoutPreemptively(Duration.ofMillis(500), () -> {
cacheService.lookup("key");
});
}
```

```
}
```

2.2 AssertJ – Fluent, Readable Assertions

AssertJ

AssertJ provides a rich fluent API for assertions. It gives much better error messages than JUnit's built-in assertions and has dedicated methods for collections, strings, dates, exceptions, and optional values. Spring Boot Test includes AssertJ by default.

AssertJ fluent assertions – strings, numbers, collections, exceptions, Optional

```
import static org.assertj.core.api.Assertions.*;

class AssertJDemo {

@Test
void stringAssertions() {
String email = "alice@example.com";
assertThat(email)
.isNotNull()
.isNotBlank()
.contains("@")
.startsWith("alice")
.endsWith(".com")
.hasSize(17)
.doesNotContain(" ");
}

@Test
void numberAssertions() {
double salary = 75000.0;
assertThat(salary)
.isPositive()
.isGreaterThan(50000)
.isLessThanOrEqualTo(100000)
.isBetween(70000.0, 80000.0);
}

@Test
void collectionAssertions() {
List<String> fruits = List.of("apple", "banana", "cherry");
assertThat(fruits)
.isNotEmpty()
.hasSize(3)
.contains("apple", "banana")
.doesNotContain("mango")
.containsExactly("apple", "banana", "cherry") // exact order
.containsExactlyInAnyOrder("cherry", "apple", "banana")
.allMatch(f -> f.length() > 3)
.anyMatch(f -> f.startsWith("a"))
```

```
.noneMatch(f -> f.isBlank());
}

@Test
void objectAssertions() {
Employee emp = new Employee(1L, "Bob", "IT", 70000.0);
assertThat(emp)
.isNotNull()
.extracting("name", "department")
.containsExactly("Bob", "IT");

// extracting with lambda
assertThat(emp)
.extracting(Employee::getSalary)
.isEqualTo(70000.0);
}

@Test
void exceptionAssertions() {
assertThatThrownBy(() -> service.findById(-1L))
.isInstanceOf(ResourceNotFoundException.class)
.hasMessageContaining("not found")
.hasMessageMatching(".*-1.*");

assertThatNoException().isThrownBy(() -> service.findById(1L));
assertThatCode(() -> service.findById(1L)).doesNotThrowAnyException();
}

@Test
void optionalAssertions() {
Optional<User> user = repo.findByEmail("alice@example.com");
assertThat(user)
.isPresent()
.hasValueSatisfying(u -> {
assertThat(u.getName()).isEqualTo("Alice");
assertThat(u.isActive()).isTrue();
});

Optional<User> missing = repo.findByEmail("nobody@example.com");
assertThat(missing).isEmpty();
}
}
```

CHAPTER 3

Parameterised & Advanced JUnit 5 Tests

@ParameterizedTest, @ValueSource, @CsvSource, @MethodSource, @RepeatedTest

3.1 @ParameterizedTest – Data-Driven Tests

@ParameterizedTest

A parameterised test is a test that runs multiple times with different input arguments. Instead of copying the same test with different values, you define it once and supply a data source. JUnit 5 supports @ValueSource, @CsvSource, @CsvFileSource, @MethodSource, @EnumSource, and @ArgumentsSource.

All parameterised test source annotations

```
// @ValueSource – single argument, simple types
@ParameterizedTest
@ValueSource(strings = {"alice@test.com", "bob@example.org", "carol@mail.net"})
@DisplayName("Valid emails should pass validation")
void validEmailsPassValidation(String email) {
assertTrue(EmailValidator.isValid(email));
}

@ParameterizedTest
@ValueSource(ints = {1, 5, 10, 100, 1000})
void positiveNumbersAreAccepted(int value) {
assertThat(calculator.square(value)).isPositive();
}

@ParameterizedTest
@NullAndEmptySource // tests null then ""
@ValueSource(strings = {" ", "\t", "\n"})
void blankStringsFailValidation(String input) {
assertFalse(validator.isValidName(input));
}

// @CsvSource – multiple arguments per test case
@ParameterizedTest(name = "{0} + {1} = {2}")
@CsvSource({
"1, 2, 3",
"10, 20, 30",
"-5, 5, 0",
"0, 0, 0"
})
void additionWorks(int a, int b, int expected) {
```

```
assertEquals(expected, calculator.add(a, b));
}

// @CsvFileSource – load data from CSV file
@ParameterizedTest
@CsvFileSource(resources = "/test-data/products.csv", numLinesToSkip = 1)
void productPricingFromCSV(String name, double price, String category) {
Product product = productService.createProduct(name, price, category);
assertThat(product.getPrice()).isEqualTo(price);
}

// @MethodSource – complex object arguments
@ParameterizedTest
@MethodSource("provideUsersForRegistration")
void registerUser(String name, String email, boolean shouldSucceed) {
if (shouldSucceed) {
assertDoesNotThrow(() -> userService.register(name, email));
} else {
assertThrows(ValidationException.class,
() -> userService.register(name, email));
}
}

static Stream<Arguments> provideUsersForRegistration() {
return Stream.of(
Arguments.of("Alice", "alice@test.com", true),
Arguments.of("Bob", "invalid-email", false),
Arguments.of("", "carol@test.com", false),
Arguments.of(null, "dave@test.com", false)
);
}

// @EnumSource – test with enum values
@ParameterizedTest
@EnumSource(value = OrderStatus.class, names = {"PENDING", "CONFIRMED"})
void activeOrdersCanBeCancelled(OrderStatus status) {
Order order = Order.builder().status(status).build();
assertTrue(orderService.canCancel(order));
}

// @RepeatedTest – run the same test N times
@RepeatedTest(value = 5, name = "Attempt {currentRepetition} of {totalRepetitions}")
void retryableOperationShouldBeIdempotent() {
String result = service.idempotentOperation("key");
assertEquals("expected-result", result);
}
```

CHAPTER 4

Mockito – Fundamentals

Mock vs Stub vs Spy, @Mock, @InjectMocks, Stubbing & Argument Matchers

4.1 What is Mockito? Core Concepts

Mock

A Mock is a fake implementation of a class or interface that records method calls and allows you to configure what it returns. Mocks replace real dependencies (database, external service, email sender) so tests are fast, deterministic, and isolated.

Type	Definition	When to Use
Mock	Completely fake object; all methods return default values unless stubbed	External dependencies (repos, services, clients)
Stub	A mock configured to return specific values – a mock with stubbing	When you care about the return value
Spy	Wraps a REAL object; real methods called unless overridden	Partial mocking; testing legacy code
Dummy	Placeholder object; method never actually called	Satisfying parameter requirements only
Fake	Simplified real implementation (e.g. in-memory repo)	Integration-style unit tests

4.2 @Mock, @InjectMocks & MockitoExtension

@Mock, @InjectMocks, @ExtendWith – complete test class

```
// Classes under test
public interface UserRepository {
Optional<User> findById(Long id);
User save(User user);
boolean existsByEmail(String email);
}
public interface EmailService {
void sendWelcomeEmail(String to, String name);
}
@Service
public class UserService {
private final UserRepository userRepository;
private final EmailService emailService;
```

```
public UserService(UserRepository userRepository, EmailService emailService) {
this.userRepository = userRepository;
this.emailService = emailService;
}

public User createUser(String name, String email) {
if (userRepository.existsByEmail(email)) {
throw new DuplicateEmailException("Email already in use: " + email);
}
User user = userRepository.save(new User(name, email));
emailService.sendWelcomeEmail(email, name);
return user;
}

public User getUserById(Long id) {
return userRepository.findById(id)
.orElseThrow(() -> new UserNotFoundException("User not found: " + id));
}
}

// ■■ Test Class ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@ExtendWith(MockitoExtension.class) // activates @Mock and @InjectMocks
class UserServiceTest {

@Mock
private UserRepository userRepository; // fake implementation

@Mock
private EmailService emailService; // fake implementation

@InjectMocks
private UserService userService; // real class; mocks injected automatically

@Test
void createUser_success() {
// ARRANGE
when(userRepository.existsByEmail("alice@test.com")).thenReturn(false);
User saved = new User(1L, "Alice", "alice@test.com");
when(userRepository.save(any(User.class))).thenReturn(saved);

// ACT
User result = userService.createUser("Alice", "alice@test.com");

// ASSERT
assertNotNull(result);
assertEquals("Alice", result.getName());
verify(emailService).sendWelcomeEmail("alice@test.com", "Alice");
}

@Test
void createUser_duplicateEmail_throwsException() {
when(userRepository.existsByEmail("alice@test.com")).thenReturn(true);
```

```
assertThrows(DuplicateEmailException.class,
() -> userService.createUser("Alice", "alice@test.com"));

verify(userRepository, never()).save(any());
verify(emailService, never()).sendWelcomeEmail(any(), any());
}
}
```

4.3 Stubbing – when / thenReturn / thenThrow

Stubbing patterns – thenReturn, thenThrow, thenAnswer, chaining

```
// Basic stubbing
when(userRepository.findById(1L))
.thenReturn(Optional.of(new User(1L, "Alice", "alice@test.com")));

when(userRepository.findById(999L))
.thenReturn(Optional.empty());

// Chained returns – different result on each call
when(stockService.getStock("ITEM-1"))
.thenReturn(10) // 1st call returns 10
.thenReturn(5) // 2nd call returns 5
.thenReturn(0); // 3rd and subsequent calls return 0

// Throw exception
when(paymentService.charge(anyDouble()))
.thenThrow(new PaymentFailedException("Card declined"));

// thenAnswer – dynamic return value based on argument
when(userRepository.save(any(User.class)))
.thenAnswer(invocation -> {
User user = invocation.getArgument(0);
user.setId(100L); // simulate DB-generated ID
return user;
});

// Consecutive stubbing shorthand
when(service.getStatus())
.thenReturn("LOADING", "LOADING", "READY");

// Stubbing with multiple conditions
when(productRepo.findByCategoryAndPriceLessThan("electronics", 1000.0))
.thenReturn(List.of(laptop, phone));
```

4.4 Argument Matchers

Matcher	Matches
any()	Any non-null object
any(Class.class)	Any non-null instance of given class
anyString()	Any non-null String

Matcher	Matches
anyInt() / anyLong() etc.	Any primitive int/long/double/boolean
anyList() / anyMap()	Any non-null List/Map
eq(value)	Exact value match (use when mixing matchers)
isNull() / notNull()	Null / non-null
contains(str)	String containing substring
startsWith(str)	String starting with value
endsWith(str)	String ending with value
matches(regex)	String matching regular expression
argThat(predicate)	Custom predicate match
intThat(predicate)	Custom int predicate
same(obj)	Same object reference (==)

Argument matchers in stubbing and verification

```
// Using argument matchers in stubbing
when(userRepository.findByNameContaining(anyString()))
.thenReturn(List.of(alice, bob));

when(orderService.createOrder(any(OrderRequest.class)))
.thenReturn(pendingOrder);

// Custom matcher with argThat
when(emailService.send(argThat(email -> email.contains("@") && email.length() > 5)))
.thenReturn(true);

// IMPORTANT: when mixing matchers, ALL arguments must use matchers
// WRONG:
// when(service.process("order-1", anyInt())).thenReturn(result);

// CORRECT: use eq() for exact values when mixing
when(service.process(eq("order-1"), anyInt())).thenReturn(result);

// Using matchers in verify()
verify(emailService).sendWelcomeEmail(eq("alice@test.com"), anyString());
verify(paymentService, times(1)).charge(anyDouble());
```

CHAPTER 5

Mockito – Advanced

@Spy, doNothing, thenAnswer, verify(), InOrder, Static Mocks & BDD Style

5.1 verify() – Interaction Verification

verify() modes and InOrder verification

```
verify(emailService).sendWelcomeEmail("alice@test.com", "Alice");

// Verification modes
verify(emailService, times(2)).send(any()); // called exactly twice
verify(emailService, atLeastOnce()).send(any()); // called >= 1 times
verify(emailService, atLeast(3)).send(any()); // called >= 3 times
verify(emailService, atMost(5)).send(any()); // called <= 5 times
verify(emailService, never()).sendPremiumOffer(any());// never called

// Verify no interactions at all
verifyNoInteractions(emailService);

// Verify no more interactions (only the ones verified)
verify(emailService).sendWelcomeEmail(any(), any());
verifyNoMoreInteractions(emailService);

// InOrder – verify calls happen in a specific sequence
@Test
void operationsShouldHappenInOrder() {
orderService.processOrder(order);

InOrder inOrder = inOrder(inventoryService, paymentService, notificationService);
inOrder.verify(inventoryService).reserve(order.getId());
inOrder.verify(paymentService).charge(order.getAmount());
inOrder.verify(notificationService).sendConfirmation(order.getCustomerId());
}
```

5.2 @Captor – Argument Capture

ArgumentCaptor

ArgumentCaptor lets you capture the actual argument passed to a mock method, so you can inspect its value in assertions. This is especially useful when the argument is a complex object created inside the method under test.

ArgumentCaptor – capturing and inspecting arguments

```
@ExtendWith(MockitoExtension.class)
class EmailNotificationTest {

@Mock private EmailGateway emailGateway;
```

```
@InjectMocks private NotificationService notificationService;

@Captor
private ArgumentCaptor<EmailMessage> emailCaptor;

@Test
void shouldSendCorrectWelcomeEmail() {
User user = new User(1L, "Alice", "alice@test.com");

notificationService.sendWelcome(user);

// Capture the EmailMessage passed to the gateway
verify(emailGateway).send(emailCaptor.capture());
EmailMessage captured = emailCaptor.getValue();

assertThat(captured.getTo()).isEqualTo("alice@test.com");
assertThat(captured.getSubject()).contains("Welcome");
assertThat(captured.getBody()).contains("Alice");
}

@Test
void shouldSendEmailToAllAdmins() {
notificationService.alertAdmins("System down");

// Capture multiple invocations
verify(emailGateway, times(3)).send(emailCaptor.capture());
List<EmailMessage> allEmails = emailCaptor.getAllValues();
assertThat(allEmails).hasSize(3);
assertThat(allEmails).allMatch(e -> e.getBody().contains("System down"));
}
}
```

5.3 @Spy – Partial Mocking

@Spy and void method stubbing

```
// @Spy wraps a REAL object – real methods run unless overridden
@ExtendWith(MockitoExtension.class)
class PricingServiceTest {

@Spy
private PricingService pricingService = new PricingService();

@Test
void testWithSpy() {
// Real method runs
double price = pricingService.calculateTax(100.0);
assertThat(price).isEqualTo(18.0); // real calculation

// Override just one method
doReturn(0.0).when(pricingService).getTaxRate();

// Now calculateTax uses the overridden rate
double zeroPriceTax = pricingService.calculateTax(100.0);
assertThat(zeroPriceTax).isEqualTo(0.0);
```

```
}
}

// void method stubbing
// doNothing (already default for void mocks, but explicit for spies)
doNothing().when(auditService).log(any());

// doThrow for void methods
doThrow(new RuntimeException("DB error")).when(auditService).log(any());

// doAnswer for void methods
doAnswer(invocation -> {
String message = invocation.getArgument(0);
System.out.println("Intercepted log: " + message);
return null;
}).when(auditService).log(anyString());
```

5.4 Mocking Static Methods (Mockito 3.4+)

Static method mocking with MockedStatic

```
<!-- pom.xml – requires mockito-inline -->
<dependency>
<groupId>org.mockito</groupId>
<artifactId>mockito-inline</artifactId>
<scope>test</scope>
</dependency>

// Static method mocking
@Test
void testStaticMethod() {
try (MockedStatic<LocalDate> mockedDate = mockStatic(LocalDate.class)) {
mockedDate.when(LocalDate::now)
.thenReturn(LocalDate.of(2025, 1, 15));

// Code that calls LocalDate.now() will get our fixed date
String result = invoiceService.generateInvoiceDate();
assertThat(result).isEqualTo("2025-01-15");
} // MockedStatic is AutoCloseable – scope ends here
}

// Mocking constructor
@Test
void testConstructor() {
UUID fixedId = UUID.fromString("11111111-1111-1111-1111-111111111111");
try (MockedConstruction<Order> mocked = mockConstruction(Order.class,
(mock, context) -> when(mock.getId()).thenReturn(fixedId))) {
Order order = new Order(); // returns mocked instance
assertThat(order.getId()).isEqualTo(fixedId);
}
}
```

5.5 BDD Mockito – given / willReturn / then

BDD Mockito – given/willReturn/then style

```
import static org.mockito.BDDMockito.*;

// BDD style mirrors Gherkin: given / when / then
@Test
void bddStyleTest() {
// GIVEN
User alice = new User(1L, "Alice", "alice@test.com");
given(userRepository.findById(1L)).willReturn(Optional.of(alice));

// WHEN
User result = userService.getUserById(1L);

// THEN
then(userRepository).should().findById(1L);
then(emailService).shouldHaveNoInteractions();
assertThat(result.getName()).isEqualTo("Alice");
}

// BDD for void methods
willDoNothing().given(auditService).log(any());
willThrow(new RuntimeException()).given(auditService).log("FATAL");

// BDD argument capture
then(emailService).should().send(emailCaptor.capture());
```

CHAPTER 6

Spring Boot Test Slices

@SpringBootTest, @WebMvcTest, @DataJpaTest, @MockBean & The Test Pyramid

6.1 The Test Pyramid & Test Slices

- **Unit Tests** (bottom, most) – Test a single class in isolation using plain JUnit + Mockito. No Spring context. Very fast.
- **Integration Tests / Slices** (middle) – Test one layer with minimal Spring context (@WebMvcTest, @DataJpaTest). Medium speed.
- **End-to-End Tests** (top, fewest) – Full Spring context + real DB (@SpringBootTest + Testcontainers). Slow but high confidence.

Annotation	Context Loaded	Spring Beans	Database	Speed
Plain JUnit + Mockito	None	None	None	Fastest
@WebMvcTest	Web layer only	@Controller, @ControllerAdvice	None	Fast
@DataJpaTest	JPA layer only	@Repository	H2 in-memory	Fast
@RestClientTest	RestTemplate/WebClient only	RestTemplate	None	Fast
@SpringBootTest	Full application context	All beans	Configured DB	Slow

6.2 @SpringBootTest – Full Integration Test

@SpringBootTest integration test with TestRestTemplate

```
// Full integration test – loads entire Spring application context
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@ActiveProfiles("test")
class UserIntegrationTest {

@Autowired
private TestRestTemplate restTemplate;

@Autowired
private UserRepository userRepository;

@MockBean // replaces real EmailService with Mockito mock in context
private EmailService emailService;

@BeforeEach
void setUp() {
userRepository.deleteAll();
```

```
}

@Test
void createUser_returnsCreatedUser() {
CreateUserRequest req = new CreateUserRequest("Alice", "alice@test.com");

ResponseEntity<User> response = restTemplate
.postForEntity("/api/users", req, User.class);

assertThat(response.getStatusCode()).isEqualTo(HttpStatus.CREATED);
assertThat(response.getBody()).isNotNull();
assertThat(response.getBody().getName()).isEqualTo("Alice");
assertThat(userRepository.count()).isEqualTo(1);
verify(emailService).sendWelcomeEmail("alice@test.com", "Alice");
}

@Test
void getUser_notFound_returns404() {
ResponseEntity<String> response = restTemplate
.getForEntity("/api/users/999", String.class);
assertThat(response.getStatusCode()).isEqualTo(HttpStatus.NOT_FOUND);
}
}

# src/test/resources/application-test.properties
spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
spring.jpa.hibernate.ddl-auto=create-drop
```

6.3 @WebMvcTest – Controller Unit Test

@WebMvcTest with MockMvc – full GET/POST/error tests

```
// @WebMvcTest loads ONLY the web layer – controllers, filters, security
// All @Service, @Repository beans must be @MockBean
@WebMvcTest(UserController.class)
class UserControllerTest {

@Autowired
private MockMvc mockMvc;

@MockBean
private UserService userService;

@Autowired
private ObjectMapper objectMapper;

@Test
void getUser_exists_returns200() throws Exception {
User alice = new User(1L, "Alice", "alice@test.com");
given(userService.getUserById(1L)).willReturn(alice);

mockMvc.perform(get("/api/users/1")
.accept(MediaType.APPLICATION_JSON))
.andExpect(status().isOk())
```

```
    .andExpect(jsonPath("$.id").value(1))
    .andExpect(jsonPath("$.name").value("Alice"))
    .andExpect(jsonPath("$.email").value("alice@test.com"));
    }

    @Test
    void createUser_validRequest_returns201() throws Exception {
    CreateUserRequest req = new CreateUserRequest("Bob", "bob@test.com");
    User saved = new User(2L, "Bob", "bob@test.com");
    given(userService.createUser("Bob", "bob@test.com")).willReturn(saved);

    mockMvc.perform(post("/api/users")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(req)))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.id").value(2))
    .andExpect(header().string("Location", containsString("/api/users/2")));
    }

    @Test
    void createUser_blankName_returns400() throws Exception {
    CreateUserRequest req = new CreateUserRequest("", "bob@test.com");

    mockMvc.perform(post("/api/users")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(req)))
    .andExpect(status().isBadRequest())
    .andExpect(jsonPath("$.message").exists());
    }

    @Test
    void getUser_notFound_returns404() throws Exception {
    given(userService.getUserById(999L))
    .willThrow(new UserNotFoundException("User not found: 999"));

    mockMvc.perform(get("/api/users/999"))
    .andExpect(status().isNotFound())
    .andExpect(jsonPath("$.status").value(404));
    }
    }
```

6.4 @DataJpaTest – Repository Test

@DataJpaTest with TestEntityManager

```
    // @DataJpaTest loads only JPA layer; uses H2 in-memory DB by default
    @DataJpaTest
    class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

    @Autowired
```

```
private TestEntityManager entityManager; // helper for test setup

@Test
void findByEmail_exists_returnsUser() {
// Arrange – persist test data
User user = new User("Alice", "alice@test.com");
entityManager.persistAndFlush(user);

// Act
Optional<User> found = userRepository.findByEmail("alice@test.com");

// Assert
assertThat(found).isPresent();
assertThat(found.get().getName()).isEqualTo("Alice");
}

@Test
void findByEmail_notExists_returnsEmpty() {
Optional<User> found = userRepository.findByEmail("nobody@test.com");
assertThat(found).isEmpty();
}

@Test
void findByDepartmentAndSalaryGreaterThan_returnsMatchingUsers() {
entityManager.persist(new User("Alice", "IT", 80000.0));
entityManager.persist(new User("Bob", "IT", 60000.0));
entityManager.persist(new User("Carol", "HR", 90000.0));
entityManager.flush();

List<User> result = userRepository
.findByDepartmentAndSalaryGreaterThan("IT", 70000.0);

assertThat(result).hasSize(1)
.extracting(User::getName)
.containsExactly("Alice");
}

@Test
void save_persistsUser() {
User user = userRepository.save(new User("Dave", "dave@test.com"));

assertThat(user.getId()).isNotNull();
assertThat(entityManager.find(User.class, user.getId())).isNotNull();
}
}
```

CHAPTER 7

MockMvc – Web Layer Testing

Complete HTTP Testing, jsonPath, Headers, Security & File Upload

7.1 MockMvc Request Building & Result Matchers

MockMvc – complete HTTP method examples with assertions

```
// MockMvc complete request builders reference

// GET with query parameters and headers
mockMvc.perform(
get("/api/products")
.param("category", "electronics")
.param("minPrice", "100")
.param("page", "0")
.header("Authorization", "Bearer " + token)
.accept(MediaType.APPLICATION_JSON)
)
.andExpect(status().isOk())
.andExpect(content().contentType(MediaType.APPLICATION_JSON))
.andExpect(jsonPath("$").isArray())
.andExpect(jsonPath("$.length()").value(3))
.andExpect(jsonPath("$[0].name").value("Laptop"))
.andExpect(jsonPath("$[0].price").value(greaterThan(500.0)))
.andExpect(jsonPath("$[*].category").value(everyItem(equalTo("electronics"))))
.andDo(print()); // print request/response to console

// POST with JSON body
mockMvc.perform(
post("/api/orders")
.contentType(MediaType.APPLICATION_JSON)
.content("""
{
"customerId": "cust-1",
"items": [{"productId": 1, "quantity": 2}],
"notes": "Please handle with care"
}
""")
)
.andExpect(status().isCreated())
.andExpect(jsonPath("$.id").isNotEmpty())
.andExpect(jsonPath("$.status").value("PENDING"))
```

```
    .andExpect(header().exists("Location"));

    // PUT
    mockMvc.perform(
    put("/api/users/{id}", 1L)
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(updateRequest))
    )
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.name").value("Updated Name"));

    // DELETE
    mockMvc.perform(delete("/api/users/{id}", 1L))
    .andExpect(status().isNoContent());

    // PATCH
    mockMvc.perform(
    patch("/api/orders/{id}/status", "order-1")
    .contentType(MediaType.APPLICATION_JSON)
    .content("""{"status":"CONFIRMED"}""")
    )
    .andExpect(status().isOk());

    // File upload
    MockMultipartFile file = new MockMultipartFile(
    "file", "avatar.jpg", MediaType.IMAGE_JPEG_VALUE,
    "fake-image-bytes".getBytes());

    mockMvc.perform(
    multipart("/api/users/1/avatar").file(file)
    )
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.avatarUrl").exists());
```

MvcResult, Security with @WithMockUser

```
    // Saving response for further assertions
    MvcResult result = mockMvc.perform(post("/api/auth/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content("""{"username":"alice","password":"secret"}"""))
    .andExpect(status().isOk())
    .andReturn();

    String responseBody = result.getResponse().getContentAsString();
    String token = JsonPath.read(responseBody, "$.accessToken");
    assertThat(token).isNotBlank();

    // Security test – unauthenticated request should return 401
    mockMvc.perform(get("/api/admin/users"))
    .andExpect(status().isUnauthorized());

    // Security test – wrong role returns 403
    @WithMockUser(username = "alice", roles = {"USER"}) // inject mock security context
```

```
@Test
void adminEndpoint_withUserRole_returns403() throws Exception {
mockMvc.perform(get("/api/admin/users"))
.andExpect(status().isForbidden());
}
@WithMockUser(username = "admin", roles = {"ADMIN"})
@Test
void adminEndpoint_withAdminRole_returns200() throws Exception {
mockMvc.perform(get("/api/admin/users"))
.andExpect(status().isOk());
}
```

CHAPTER 8

Repository & Database Testing

@DataJpaTest, TestEntityManager, Testcontainers & @Sql

8.1 Testcontainers – Real Database in Tests

Testcontainers

Testcontainers is a Java library that provides lightweight, throwaway Docker containers for integration tests. Instead of an H2 in-memory database that behaves differently from production PostgreSQL/MySQL, Testcontainers spins up a real database container for your tests and shuts it down when done.

Testcontainers with PostgreSQL – real database integration test

```
@SpringBootTest
@Testcontainers
@ActiveProfiles("test")
class UserRepositoryIntegrationTest {

// Starts a real PostgreSQL container
@Container
static PostgreSQLContainer<?> postgres =
new PostgreSQLContainer<>("postgres:16-alpine")
.withDatabaseName("testdb")
.withUsername("test")
.withPassword("test")
.withInitScript("schema.sql"); // runs DDL on startup

// Dynamically configure Spring datasource from container
@DynamicPropertySource
static void configureProperties(DynamicPropertyRegistry registry) {
registry.add("spring.datasource.url", postgres::getJdbcUrl);
registry.add("spring.datasource.username", postgres::getUsername);
registry.add("spring.datasource.password", postgres::getPassword);
}

@Autowired
private UserRepository userRepository;

@BeforeEach
void setUp() { userRepository.deleteAll(); }

@Test
void findByEmailWithPostgreSQL() {
userRepository.save(new User("Alice", "alice@test.com"));

Optional<User> user = userRepository.findByEmail("alice@test.com");
```

```
assertThat(user).isPresent();
assertThat(user.get().getName()).isEqualTo("Alice");
}

@Test
void testPostgreSQLSpecificQuery() {
// Test PostgreSQL-specific features (e.g. full-text search, JSONB)
userRepository.save(new User("Alice Smith", "alice@test.com"));

List<User> results = userRepository.searchByFullText("Alice");
assertThat(results).hasSize(1);
}
}
```

@Sql annotation for data setup

```
// @Sql – execute SQL scripts before/after tests
@DataJpaTest
class OrderRepositoryTest {

@Autowired
private OrderRepository orderRepository;

@Test
@Sql("/sql/insert-test-orders.sql") // runs BEFORE test
@Sql(scripts = "/sql/cleanup.sql",
executionPhase = Sql.ExecutionPhase.AFTER_TEST_METHOD) // runs AFTER
void findOrdersByCustomer_returnsOrders() {
List<Order> orders = orderRepository.findByCustomerId("cust-1");
assertThat(orders).hasSize(3);
}

// Class-level @Sql applies to all methods
@Test
@Sql(scripts = {"/sql/insert-customers.sql", "/sql/insert-orders.sql"})
void multipleScripts() {
assertThat(orderRepository.count()).isGreaterThan(0);
}
}

# /sql/insert-test-orders.sql
INSERT INTO orders (id, customer_id, status, total) VALUES
('ORD-1', 'cust-1', 'PENDING', 100.00),
('ORD-2', 'cust-1', 'CONFIRMED', 250.00),
('ORD-3', 'cust-1', 'CANCELLED', 50.00);
```

CHAPTER 9

WireMock – External Service Testing

Stubbing HTTP Endpoints, Request Matching & Testing Resilience

9.1 What is WireMock?

WireMock

WireMock is a library for stubbing and mocking HTTP services. When your microservice calls an external REST API (payment gateway, shipping service, third-party API), WireMock lets you simulate that API's responses in tests – including error scenarios, slow responses, and specific status codes – without needing the real service.

WireMock – stub HTTP, test circuit breaker, test timeouts

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureWireMock(port = 0) // random port, set in application-test.properties
class PaymentServiceTest {

@Autowired
private PaymentService paymentService;

@Test
void processPayment_success() {
// Stub the external payment gateway
stubFor(post(urlEqualTo("/payments/charge"))
.withHeader("Content-Type", equalTo("application/json"))
.withRequestBody(matchingJsonPath("$.amount", equalTo("100.0")))
.willReturn(aResponse()
.withStatus(200)
.withHeader("Content-Type", "application/json")
.withBody("""
{"paymentId":"PAY-123","status":"SUCCESS","transactionRef":"TXN-456"}
""")));

PaymentResult result = paymentService.charge("cust-1", 100.0);

assertThat(result.getStatus()).isEqualTo("SUCCESS");
assertThat(result.getPaymentId()).isEqualTo("PAY-123");

// Verify the call was actually made
verify(postRequestedFor(urlEqualTo("/payments/charge")));
}

@Test
void processPayment_gatewayDown_circuitBreakerActivates() {
// Simulate service unavailable
```

```
stubFor(post(urlEqualTo("/payments/charge"))
.willReturn(aResponse().withStatus(503)));

// After 5 failures, circuit breaker should open
for (int i = 0; i < 5; i++) {
paymentService.charge("cust-1", 100.0);
}

// Circuit is now open – verify fallback is returned
PaymentResult result = paymentService.charge("cust-1", 100.0);
assertThat(result.getStatus()).isEqualTo("FALLBACK");
}

@Test
void processPayment_slowResponse_timeoutFallback() {
// Simulate a slow gateway (3 second delay)
stubFor(post(urlEqualTo("/payments/charge"))
.willReturn(aResponse()
.withStatus(200)
.withFixedDelay(3000))); // 3 second delay

// Our timeout is 2 seconds, should trigger fallback
PaymentResult result = paymentService.charge("cust-1", 100.0);
assertThat(result.getStatus()).isEqualTo("TIMEOUT_FALLBACK");
}
}

# application-test.properties
payment.gateway.url=http://localhost:${wiremock.server.port}
```

CHAPTER 10

Test Best Practices & Code Coverage

AAA Pattern, Naming Conventions, JaCoCo, Mutation Testing & Anti-Patterns

10.1 AAA Pattern – Arrange, Act, Assert

Perfect AAA-structured test

```
// Every test follows the AAA pattern
@Test
@DisplayName("Creating order with valid items should return PENDING order")
void createOrder_withValidItems_returnsPendingOrder() {

// ■■ ARRANGE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Set up test data, configure mocks, prepare expected values
String customerId = "cust-123";
List<OrderItem> items = List.of(
new OrderItem("prod-1", 2, 29.99),
new OrderItem("prod-2", 1, 49.99))
);
given(productService.validateStock(any())).willReturn(true);
given(orderRepository.save(any())).willAnswer(inv -> {
Order o = inv.getArgument(0);
o.setId(UUID.randomUUID().toString());
return o;
});

// ■■ ACT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Execute the single action being tested
Order result = orderService.createOrder(customerId, items);

// ■■ ASSERT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Verify the expected outcome
assertThat(result).isNotNull();
assertThat(result.getId()).isNotBlank();
assertThat(result.getStatus()).isEqualTo(OrderStatus.PENDING);
assertThat(result.getTotalAmount()).isEqualTo(109.97);
then(productService).should().validateStock(items);
then(orderRepository).should().save(any(Order.class));
}
```

10.2 JaCoCo – Code Coverage

JaCoCo coverage plugin with minimum thresholds

```
<!-- pom.xml – JaCoCo plugin -->
<plugin>
<groupId>org.jacoco</groupId>
<artifactId>jacoco-maven-plugin</artifactId>
<version>0.8.11</version>
<executions>
<execution>
<id>prepare-agent</id>
<goals><goal>prepare-agent</goal></goals>
</execution>
<execution>
<id>report</id>
<phase>verify</phase>
<goals><goal>report</goal></goals>
</execution>
<execution>
<id>check</id>
<goals><goal>check</goal></goals>
<configuration>
<rules>
<rule>
<element>BUNDLE</element>
<limits>
<limit>
<counter>LINE</counter>
<value>COVEREDRATIO</value>
<minimum>0.80</minimum> <!-- 80% line coverage minimum -->
</limit>
<limit>
<counter>BRANCH</counter>
<value>COVEREDRATIO</value>
<minimum>0.70</minimum>
</limit>
</limits>
</rule>
</rules>
</configuration>
</execution>
</executions>
</plugin>

# Run: mvn verify
# Report: target/site/jacoco/index.html
```

10.3 Testing Anti-Patterns

Anti-Pattern	Problem	Correct Approach
Testing implementation details	Brittle – breaks when internals change even if behaviour is correct	Test public behaviour, not private methods
Too many assertions per test	Hard to know WHAT failed when the test breaks	One logical assertion per test; use assertAll
Mocking everything	Tests pass even when integration is broken	Use real objects where fast; mock only I/O boundaries
Asserting on mock interactions only	Doesn't verify actual output/state	Assert return values AND verify side effects
Sharing mutable test state	Test order dependency; flaky tests	@BeforeEach creates fresh instances for each test
Over-broad argument matchers	any() everywhere hides real contract	Use specific matchers; eq() for important args
Ignoring exception message	Exception type alone is insufficient	Assert exception message content too
@SpringBootTest for unit tests	100x slower; loads full context unnecessarily	Plain JUnit + Mockito for unit tests
Production code modified for tests	Indicates design problem; not testable	Refactor to use dependency injection

+ Follow the Test Pyramid: many small fast unit tests (JUnit+Mockito), fewer medium integration tests (@WebMvcTest, @DataJpaTest), and a handful of full E2E tests (@SpringBootTest+Testcontainers). This gives maximum confidence with minimum run time.

! Avoid using @SpringBootTest for tests that only need @WebMvcTest or @DataJpaTest. Loading the full context makes tests 5-10x slower. Use the most targeted slice available.

Quick Reference – All Testing Annotations

Annotation / Class	Layer	Purpose
@Test	JUnit 5	Marks a method as a test
@BeforeEach / @AfterEach	JUnit 5	Setup/teardown before/after each test
@BeforeAll / @AfterAll	JUnit 5	Setup/teardown once per class (static)
@DisplayName	JUnit 5	Human-readable test name
@Disabled	JUnit 5	Skips a test
@Nested	JUnit 5	Groups tests in inner class
@ParameterizedTest	JUnit 5	Runs test with multiple data sets
@ValueSource / @CsvSource	JUnit 5	Provides inline test data
@MethodSource	JUnit 5	Provides test data from a factory method
@RepeatedTest	JUnit 5	Repeats a test N times
@ExtendWith	JUnit 5	Registers JUnit extension (Mockito, Spring)
@Mock	Mockito	Creates a Mockito mock
@Spy	Mockito	Creates a partial Mockito spy
@InjectMocks	Mockito	Creates class under test; injects mocks
@Captor	Mockito	Creates an ArgumentCaptor
@MockitoExtension	Mockito	Activates @Mock/@InjectMocks annotations
@SpringBootTest	Spring	Loads full application context
@WebMvcTest	Spring	Loads only web layer (controllers)
@DataJpaTest	Spring	Loads only JPA layer (repositories)
@RestClientTest	Spring	Tests RestTemplate/WebClient calls
@MockBean	Spring	Adds Mockito mock to Spring context
@SpyBean	Spring	Adds Mockito spy to Spring context
@Sql	Spring	Runs SQL scripts before/after tests
@WithMockUser	Spring Sec	Injects mock security user in context
@Testcontainers / @Container	Testcontainers	Manages Docker containers for tests
@DynamicPropertySource	Spring	Overrides properties from container config
@AutoConfigureWireMock	WireMock	Sets up WireMock server for HTTP stubbing